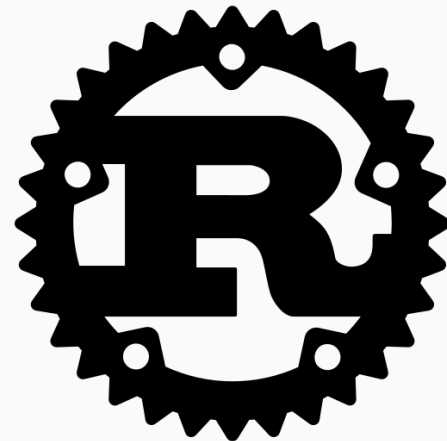


Biblioteca extensible de optimización metaheurística en **Rust**



Realizado por: David Muñoz del Valle

Tutorizado por: Gabriel Jesús Luque Polo

Convocatoria Julio 2026

Estructura de la Presentación

01 ¿Qué son las Metaheurísticas?

Introducción a los métodos de optimización aproximados.

02 ¿Por qué Rust?

Eficiencia, seguridad de memoria y ecosistema del lenguaje.

03 Metodología Seguida

Desarrollo ágil y control de hitos con Jira.

04 Arquitectura Obtenida

Diseño modular: algoritmos, operadores y soluciones.

05 Experimentos y Resultados

Evaluación de rendimiento entre diferentes bibliotecas..

06 Implementaciones Extras

Report, puntos de restauración y carga de problemas desde archivos.

07 Conclusiones y Líneas Futuras

Conclusiones del trabajo y propuestas de desarrollo futuro.

¿Qué son las Metaheurísticas?

Exactos vs Aproximados

MÉTODOS APROXIMADOS

Sacrifican la optimalidad absoluta por soluciones de muy alta calidad en tiempos de cómputo razonables.

¿Cuándo usarlos?

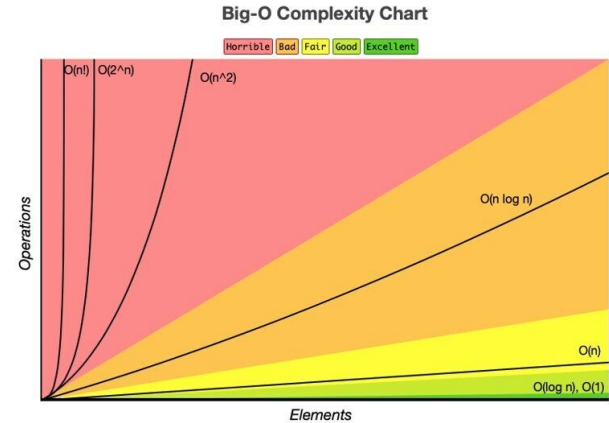
Problemas de **alta complejidad** a gran escala, donde un método exacto tardaría un tiempo inasumible.

MÉTODOS EXACTOS

Garantizan encontrar la solución óptima global, pero su coste computacional puede crecer exponencialmente y ser inviable para casos reales.

¿Cuándo usarlos?

Problemas de **baja complejidad** o de escala menor, donde el **óptimo absoluto es imprescindible**.



¿Qué son las Metaheurísticas?

Etimología y Principios Clave

Heurística: εὕρισκειν

Del griego «encontrar», «descubrir» o «hallar».

Metaheurística: Meta + Heurística

Combina el prefijo griego **meta-** («más allá de»). Representa una estrategia de **nivel superior**.



El Equilibrio Fundamental del Espacio de Búsqueda

Exploración

Permite investigar **nuevas regiones** del espacio de búsqueda de forma global, garantizando la diversidad y evitando que el algoritmo quede atrapado en óptimos locales subóptimos.

Explotación

Se centra en **refinar y explotar** las mejores soluciones encontradas hasta el momento de forma local, intensificando la búsqueda para aproximarse al óptimo definitivo.

¿Qué son las Metaheurísticas?

Ejemplo: Algoritmo Genético

Inspirado en la evolución natural. En lugar de realizar una búsqueda exhaustiva, este algoritmo mantiene y evoluciona una población de soluciones generación tras generación.



1. Inicialización

Genera la población inicial de soluciones, comúnmente de forma aleatoria o bajo criterios específicos del problema.

2. Evaluación

Calcula la calidad (fitness) de cada solución individual mediante una función objetivo de rendimiento.

3. Selección

Escoge los individuos más aptos para reproducirse, favoreciendo la supervivencia de las mejores soluciones.

4. Cruce (Crossover)

Combina la información de dos soluciones progenitoras para generar descendencia con rasgos heredados de ambas.

5. Mutación

Aplica pequeñas variaciones aleatorias para inyectar diversidad en la población y escapar de óptimos locales.

6. Reemplazo

Determina qué soluciones permanecen en el grupo y cuáles son sustituidas por las nuevas generaciones.

¿Qué son las Metaheurísticas?

Trayectoria vs Población



Algoritmos de Trayectoria

Trabajan sobre una **única solución** que va evolucionando y moviéndose por el espacio de búsqueda (ej. Enfriamiento Simulado, Búsqueda Tabú).



Algoritmos de Población

Mantienen y hacen evolucionar un **conjunto de soluciones** simultáneamente, fomentando el intercambio de información (ej. Algoritmos Genéticos).

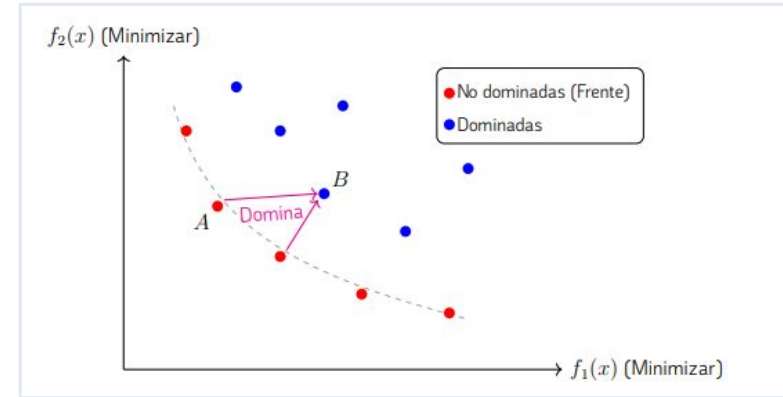
¿Qué son las Metaheurísticas?

Optimización Multi-Objetivo

EL DESAFÍO MULTI-OBJETIVO

En el mundo real, los problemas presentan múltiples objetivos en conflicto directo (ej. maximizar rendimiento y minimizar coste).

- **Dominancia de Pareto:** Al desaparecer el concepto de solución única, se adopta el criterio de dominancia para identificar soluciones preferibles.
- **Aproximación del Frente:** Las metaheurísticas buscan aproximar el conjunto de mejores compromisos posibles (soluciones no dominadas).



¿Por qué Rust?

Ventajas Clave del Lenguaje

EFICIENCIA Y SEGURIDAD

- **Rendimiento Extremo:** Rust no emplea un recolector de basura (Garbage Collector) ni un intérprete, eliminando las pausas de ejecución indeseadas.
- **Abstracciones de Coste Cero:** Facilita el desarrollo con abstracciones de alto nivel que se compilan a código máquina altamente optimizado.
- **Garantías de Memoria en Compilación:** El revolucionario sistema de propiedad (**Ownership**) y el estricto analizador de vidas (**Lifetimes**) impiden accesos inválidos, use-after-free o data-races.
- **Concurrencia Segura inherente:** El compilador asegura la ausencia de condiciones de carrera de datos a nivel de memoria.

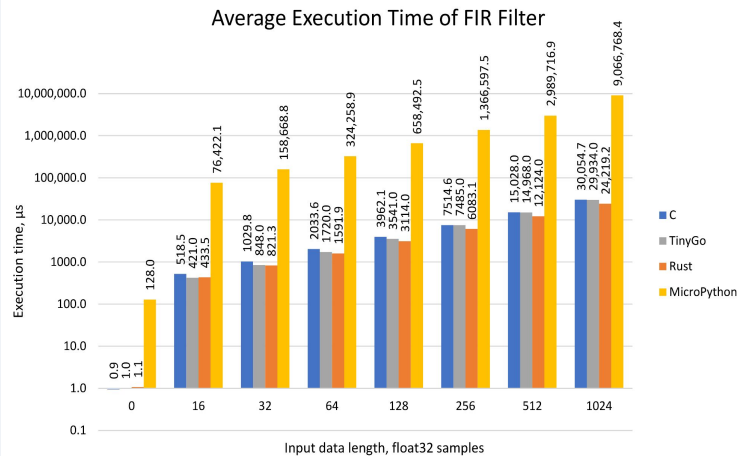
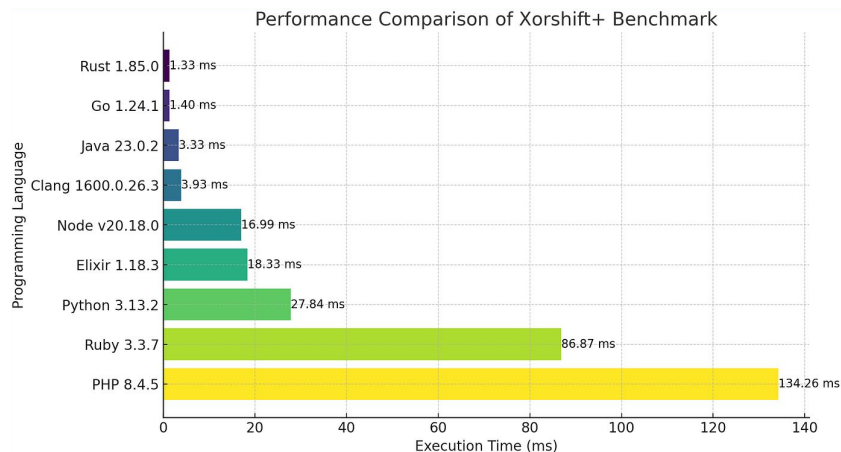


Cargo

El gestor de paquetes de Rust

¿Por qué Rust?

Rendimiento Comparativo



¿Por qué Rust?

Crecimiento y Popularidad

CRECIMIENTO DE DESARROLLADORES

Rust y Go crecen a un ritmo superior al de la población global de desarrolladores (Comparativa de cambio en total de usuarios 2024 vs 2022).

Rust +100%

Go +57%

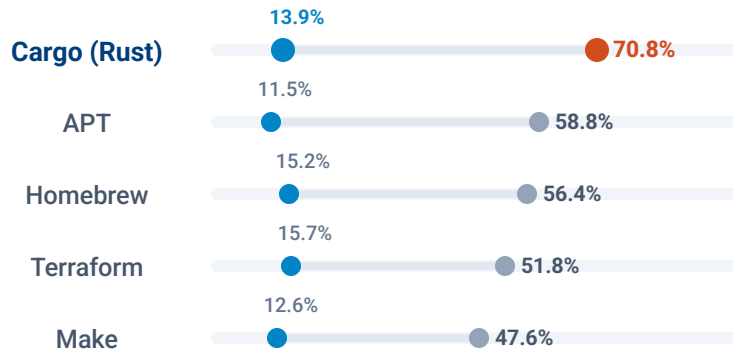
JavaScript +39%

Evolución de desarrolladores +39%

Cargo es la herramienta de desarrollo e infraestructura cloud más valorada en 2025 (Encuesta Stack Overflow)

■ Deseada

■ Admirada



Fases del Proyecto y Control de Calidad

01

Planificación y Diseño

Fase Inicial: Definición de los requisitos de la biblioteca de optimización y de la planificación de los diferentes hitos.

02



Desarrollo e Iteración

Ciclo Ágil con Jira: Flujo continuo de trabajo coordinado con el tutor. Las tareas se planifican en Jira (**Software de Gestión de Proyecto**), se desarrollan incrementalmente y se someten a revisión sistemática antes de su integración final.

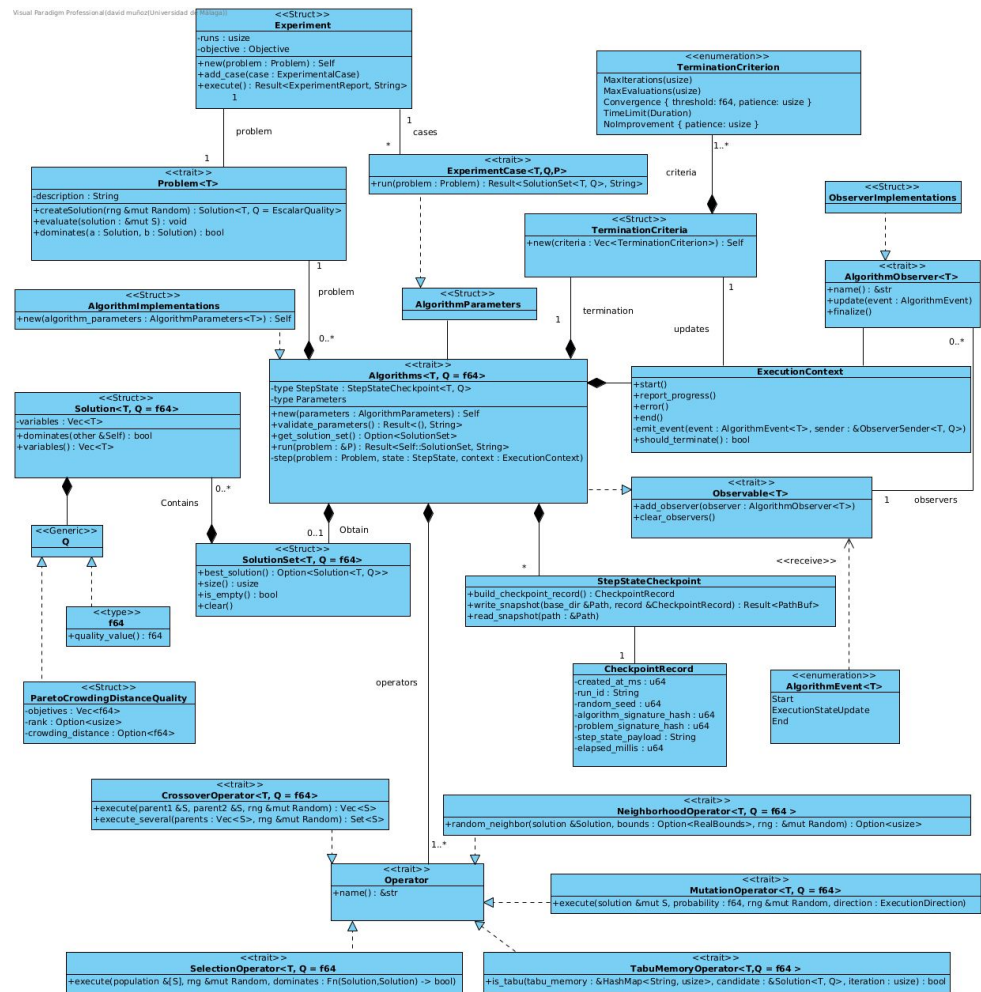
03

Validación y Experimentación

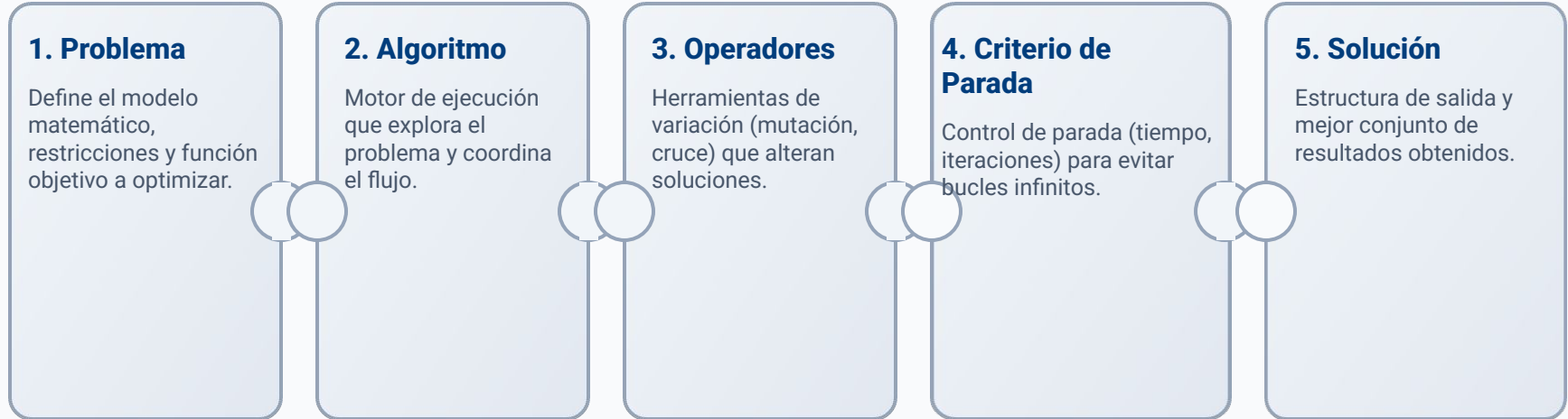
Fase de Cierre: Ejecución de experimentos comparativos contra **bibliotecas consolidados** ante problemas clásicos (como TSP).

Arquitectura Obtenida

Diagrama UML



Módulos y Sus Responsabilidades



Problemas



01

Restricciones

Definición: Contienen las restricciones y especificaciones del modelo matemático del problema.

02

Solución Inicial

Generación: Crean las soluciones iniciales de los problemas para iniciar la optimización.

03

Evaluación

Cálculo: Son los encargados de evaluar cada una de las soluciones obtenidas.

04

Comparación

Decisión: Son los encargados de comparar las soluciones para seleccionar la mejor.

Algoritmos



01

Ejecución

Resultado: Son los encargados de las ejecuciones del algoritmo y devuelven el resultado final.

02

Observadores

Ciclo de vida: Controlan de manera directa el ciclo de vida de los observadores del sistema.

03

Snapshots

Historial: Encargados de realizar y gestionar las capturas del estado actual.

04

Parámetros

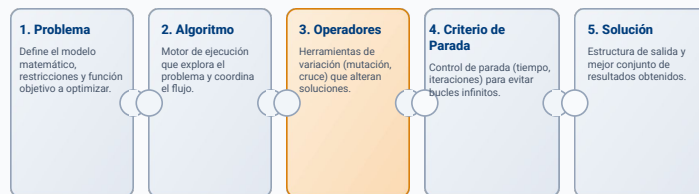
Validación: Son los responsables de validar adecuadamente todos los parámetros de entrada.

05

Operadores

Selección: Encargados de seleccionar los operadores con los que ejecutar el algoritmo.

Operadores



01

Mutación

Modifica soluciones de forma aleatoria para explorar nuevas áreas.

02

Selección

Elige las mejores soluciones para la siguiente generación.

03

Cruce

Combina dos soluciones para generar descendencia con mejores rasgos.

04

Vecindad

Explora alternativas cercanas para afinar las soluciones actuales.

05

Memoria

Guarda el histórico para evitar repeticiones y guiar la búsqueda.

Criterio De Parada



01

Máx Iteraciones

Iteraciones: Número máximo de pasos permitidos.

02

Máx Evaluaciones

Evaluaciones: Límite en el número total de evaluaciones de la función objetivo.

03

Convergencia

Convergencia:
Parada si el cambio relativo es menor que un umbral durante iteraciones consecutivas.

04

Tiempo

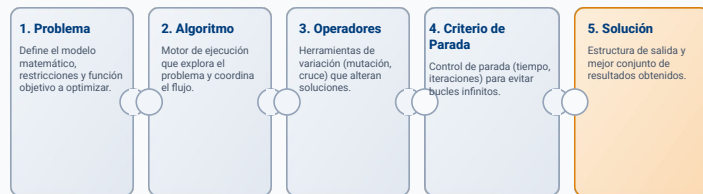
Límite de Tiempo:
Duración máxima de tiempo real (wall-clock) asignada para la ejecución.

05

No Mejora

Sin Mejora: Parada cuando la calidad de la solución no mejora tras un número de iteraciones.

Solución



Diseño Minimalista y de Alta Eficiencia

La solución es la unidad fundamental con la que opera la biblioteca de manera ininterrumpida. Su estructura ligera evita sobrecargas que comprometan el rendimiento temporal y el consumo de memoria.

■ Desafío de Soporte Dual

Soporta simultáneamente entornos mono-objetivo (un único valor escalar de aptitud) y enfoques de optimización multi-objetivo.

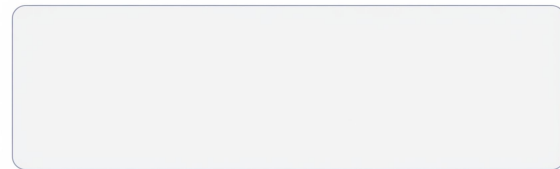
■ Gestión Multi-Objetivo Avanzada

Administra de forma estructurada vectores de funciones objetivo, rangos de dominancia y métricas de densidad como la distancia de apilamiento.

Values



Fitness (Optional)



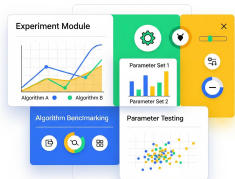
Otros módulos

01

Módulo de Experimentos

Buscar el mejor algoritmo para un problema

Diseñado para ejecutar comparativas entre múltiples algoritmos o variando parámetros clave.



02

Módulo de Utilidades

Garantía Zero-Dependency

Sistemas que reemplazan dependencias externas:

- Números pseudoaleatorios
- Gestión de rutas
- Hashing y Benchmarks



03

Módulo de Observables

Seguimiento y Métricas

Sistemas para registrar y observar la evolución del algoritmo:

- Métricas de aptitud en tiempo real
- Progreso detallado de iteraciones
- Estado de convergencia



Algoritmos Basados en Pasos



Bibliotecas de Referencia y Entorno

Aislamiento de Entorno

Contenedores Linux: Garantizan la reproducibilidad de los experimentos al aislar la ejecución de las bibliotecas.



Eficiencia y Optimización C++ (pagmo)

Computación Paralela: Basada en C++, actúa como el estándar de máximo rendimiento y optimización multihilo para la resolución de problemas de optimización global.



Ecosistema Java & Python

jMetal / jMetalPy: Plataformas consolidadas en la literatura para optimización metaheurística multi-objetivo en arquitecturas orientadas a objetos e integración ágil.

Cómputo Científico Python

SciPy Suite: Conjunto de algoritmos matemáticos y matemáticamente optimizados que sirven de base de contraste para la robustez y convergencia del sistema.

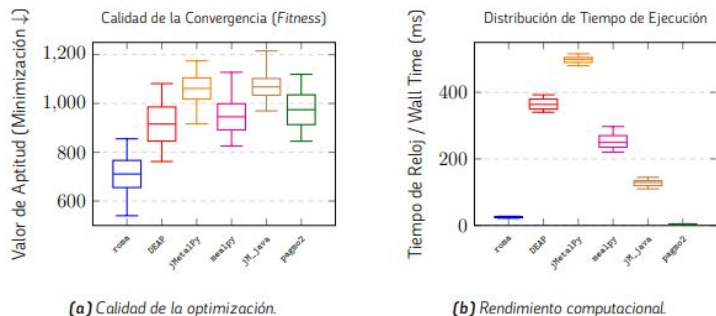


Frameworks de Metaheurísticas

mealpy: Biblioteca moderna de Python centrada en algoritmos bio-inspirados de última generación, evaluando la extensibilidad del diseño propuesto.



Protocolo de Evaluación de 3 Fases



Ejemplo del resultado de un experimento realizado sobre la función de Rastrigin mediante Hill Climbing

01 Calidad de Solución

Convergencia: Usando varias semillas obtenemos el valor de aptitud (mono-objetivo) o Hipervolumen 2D (multi-objetivo) analizados mediante diagramas de caja.

02 Rendimiento

Eficiencia CPU: Registro micrométrico del tiempo de muro (wall time) y CPU para auditar la ventaja de Rust ante entornos virtuales.

03 Significancia

Validación Estadística: Test de Friedman y análisis post-hoc (Nemenyi/Holm) para generar matrices de comparación por pares.

Validación y Significancia Estadística

0
1

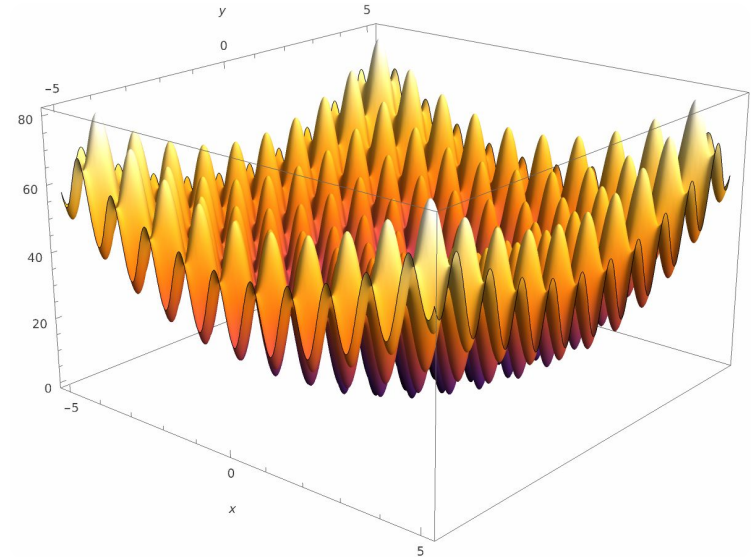
Contraste Global

Test de Friedman: Evaluación **no paramétrica** global para determinar la existencia de diferencias estadísticamente significativas en el grupo.

0
2

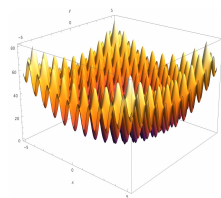
Pruebas Post-Hoc

Nemenyi y Holm: Comparación múltiple por pares que identifica qué algoritmos difieren entre sí, controlando la tasa de error acumulado.



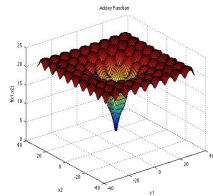
Visualización de la función de Rastrigin

Experimentos Realizados



Función de Rastrigin

Optimización continua mono-objetivo altamente multimodal, ideal para evaluar la **evasión de óptimos locales**.



Función de Ackley

Superficie de búsqueda **compleja** y altamente rugosa utilizada para medir la resistencia de los algoritmos ante ruido.



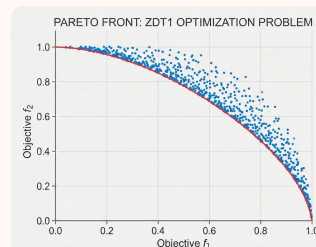
Problema de la mochila

Optimización combinatoria 0/1 con restricción de peso para maximizar el valor de los elementos seleccionados y manejar la **factibilidad** de estos.



Problema del Viajante

Optimización **combinatoria** (TSP-48) para encontrar la ruta óptima que recorre 48 ciudades de manera eficiente.



Problema ZDT1

Problema sencillo de Optimización multi-objetivo con frontera de Pareto convexa.

TSP-48: Evaluación Combinatoria con Presupuesto Temporal

Escenario Experimental Problema Agente Viajero: Instancia **tsp_48_clustered_euc2d** (48 nodos clúster 2D). Algoritmo Genético clásico con límite estricto de **5.0s** de ejecución por semilla (28 ejecuciones). El rendimiento mide la eficiencia combinatoria bruta de la arquitectura.

Métricas de Calidad de Solución (Minimización de la Longitud de Ruta y Eficiencia)

pagmo2_cpp Líder global absoluto	Mejor Ruta 1022.0	Mediana 1251.0	Evaluaciones 3 220 000	Peor 1644.0	Dispersión IQR: 171.3
roma (Propuesta) Empate técnico con C++	Mejor Ruta 1089.0	Mediana 1298.5	Evaluaciones 3 180 000	Peor 1547.0	Dispersión IQR: 173.3
DEAP Vectorizado con C	Mejor Ruta 1247.0	Mediana 1440.0	Evaluaciones 132 347	Peor 1738.0	Dispersión IQR: 175.3
jMetal_java Penalizado por JVM	Mejor Ruta 1364.0	Mediana 1661.5	Evaluaciones 4 740 000	Peor 2239.0	Dispersión IQR: 230.3
jMetalPy Sobrecarga GIL	Mejor Ruta 2652.0	Mediana 3276.5	Evaluaciones 77 791	Peor 3965.0	Dispersión IQR: 452.0
mealpy Lentitud VM	Mejor Ruta 4801.0	Mediana 5198.0	Evaluaciones ?*	Peor 5493.0	Dispersión IQR: 277.3

Incapacidad para obtener el número de evaluaciones en mealpy*: La API pública de esta biblioteca no nos permite obtener el número de evaluaciones realizadas durante la ejecución

TSP-48: Análisis Estadístico (GA)

Algoritmo	roma	DEAP	jMetal	jMetalPy	mealpy
pagmo2_cpp	0.9997	0.3023	< 0.001	< 0.001	< 0.001
roma	-	0.4750	0.0012	< 0.001	< 0.001
DEAP	-	-	0.2652	< 0.001	< 0.001
jMetal	-	-	-	0.1463	< 0.001
jMetalPy	-	-	-	-	0.3422
mealpy	-	-	-	-	-

Nota: Los valores en naranja y negrita ($p < 0.05$) indican una diferencia estadísticamente significativa de acuerdo al Test de Nemenyi.

INTERPRETACIÓN DE SIGNIFICANCIA ESTADÍSTICA

La propuesta **roma (Rust)** muestra un comportamiento equivalente al líder absoluto **pagmo2 (C++)** con un p-value de **0.9997**, superando significativamente ($p < 0.05$) a todos los frameworks basados en Java y Python (jMetal, jMetalPy y mealpy).

Función ZDT1: Evaluación Multi-Objetivo con NSGA-II

Escenario Experimental: Problema continuo **ZDT1 (D = 30)** resuelto con el algoritmo evolutivo **NSGA-II** (25 000 evaluaciones por ejecución, 28 ejecuciones). La calidad se evalúa mediante la métrica del **Hipervolumen 2D** con punto de referencia [1.1, 10.1].

Tabla de Resultados (Maximización): Calidad de Solución vs. Rendimiento Computacional

roma (Propuesta) Líder absoluto en tiempo y consistencia	Mejor Aptitud 10.7706	Mediana Aptitud 10.7700	Peor Aptitud 10.7694	Mediana Muro (ms) 175.94	Rango de Tiempo (ms) 160.0 – 182.18
pagmo2_cpp Rápido pero con fallos de validación	Mejor Aptitud 10.7699	Mediana Aptitud 10.7694	Peor Aptitud 10.7683	Mediana Muro (ms) 158.35	Rango de Tiempo (ms) 150.0 – 168.24
jMetal_java Rendimiento intermedio JVM	Mejor Aptitud 10.7692	Mediana Aptitud 10.7687	Peor Aptitud 10.7678	Mediana Muro (ms) 736.27	Rango de Tiempo (ms) 700.0 – 844.56
jMetalPy Penalizado por sobrecarga GIL	Mejor Aptitud 10.7696	Mediana Aptitud 10.7691	Peor Aptitud 10.7682	Mediana Muro (ms) 3793.49	Rango de Tiempo (ms) 3700.0 – 3887.51
DEAP Vectorizado con C pero lento	Mejor Aptitud 10.7699	Mediana Aptitud 10.7692	Peor Aptitud 10.7687	Mediana Muro (ms) 4411.62	Rango de Tiempo (ms) 4300.0 – 4549.36

Análisis Crítico: roma demuestra un rendimiento extremo al optimizar el cuello de botella crítico de NSGA-II (ordenación no dominada). Ofrece la mayor precisión numérica y consistencia, superando hasta en 25 veces el tiempo de ejecución de las alternativas en Python con un consumo temporal prácticamente equivalente a la implementación pura de C++.

ZDT1: Análisis Estadístico (NSGA-II)

Algoritmo	roma	DEAP	jMetalPy	pagmo2*	jMetal
roma (Rust)	-	0.0001	< 0.001	0.0115	< 0.001
DEAP (Py)	-	-	0.9764	0.7611	0.0003
jMetalPy (Py)	-	-	-	0.3883	0.0035
pagmo2* (C++)	-	-	-	-	< 0.001
jMetal (Java)	-	-	-	-	-

Nota: Los valores en naranja y negrita ($p < 0.05$) indican una diferencia estadísticamente significativa de acuerdo al Test de Nemenyi.

*pagmo2 arrojó fallos de validación topológica previos en el frente de Pareto.

INTERPRETACIÓN DE SIGNIFICANCIA ESTADÍSTICA Y CONCLUSIONES

Tras rechazar la hipótesis nula mediante la prueba de Friedman, el análisis post-hoc confirma que la implementación **roma (Rust)** no solo alcanza una velocidad sin precedentes (175.94 ms), sino que supera de forma robusta y estadísticamente significativa a todas las alternativas de Java y Python.

Características y Capacidades Adicionales de la Biblioteca

01

Reportes en HTML

Visualización Activa: Generación automática de reportes dinámicos al finalizar la ejecución, permitiendo analizar curvas de convergencia y estadísticas clave sin necesidad de herramientas externas.

02

Puntos de Restauración

Checkpoints en Disco: Persistencia periódica del estado del algoritmo (**roma**) para reanudar de forma segura ejecuciones largas ante fallos o interrupciones del sistema.

03

Carga Extensible

Formatos Estándar: Importación directa de problemas y conjuntos de datos mediante archivos estructurados, garantizando interoperabilidad y flexibilidad absoluta.

Reportes en HTML: Visualización sencilla de datos

Observadores HTML

Generación automática al finalizar la ejecución, sin dependencias externas.

Componentes Clave:

- Curva de Evolución:**

Progreso en tiempo real de fitness (Mejor, Medio y Peor).

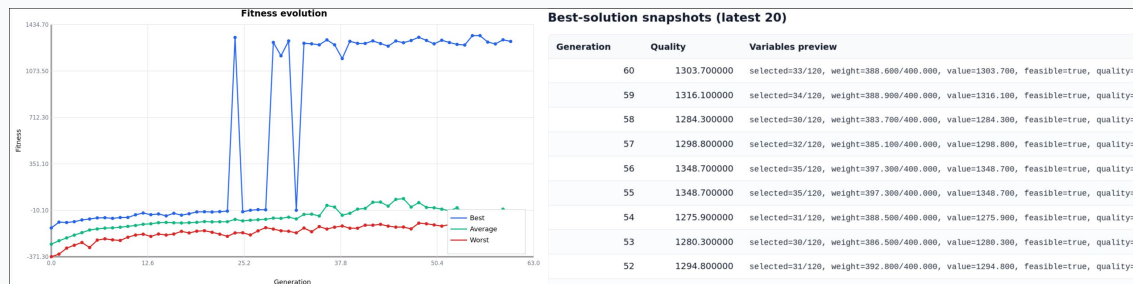
- Historial de Soluciones:**

Tabla interactiva con el historial de las últimas generaciones.

- Mejor Individuo:**

Desglose completo de variables del mejor resultado factible.

Componentes del Reporte Generado

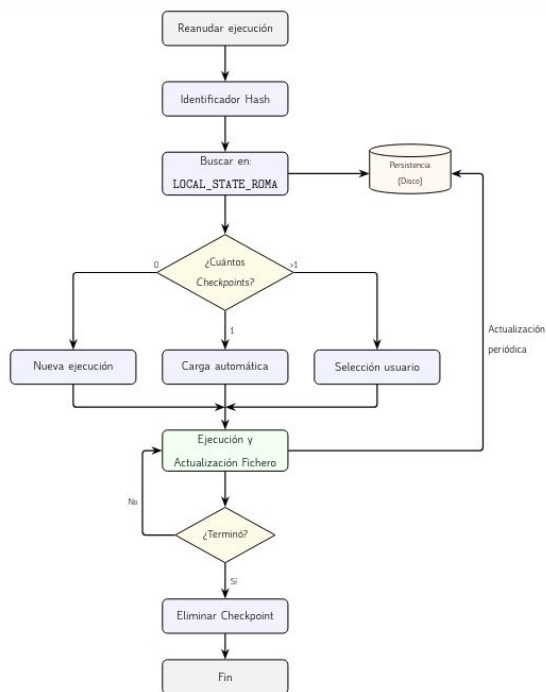


Best solution found

Generation	Quality	Variables preview
60	1303.700000	selected=33/120, weight=388.600/400.000, value=1303.700, feasible=true, quality=1303.700

- **Curva de Fitness (Izquierda):** Muestra visualmente la velocidad de convergencia y estabilidad del algoritmo.
- **Historial de Snapshots (Derecha):** Tabla ordenada de calidad por generación para auditar la evolución.
- **Detalle de Solución (Abajo):** Inspección rápida de variables de decisión, peso, valor y factibilidad.

Puntos de Restauración: Resiliencia y Checkpoints en Disco



Flujo de Restauración y Selección Interactiva

```

david ~/roma/roma/examples cargo run --example knapsack_hc_demo -- --resume
Compiling roma v0.1.0 (/home/david/roma/roma)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.08s
Running `/home/david/roma/roma/target/debug/examples/knapsack_hc_demo --resume`
  
```

--- CHECKPOINT SELECTION ---

ID	AGE	ELAPSED	INFO
[1]	3 days ago	00:00:00	> iter=59750;eval=59751;seed=8238046302331804803;
[2]	3 days ago	00:00:00	> iter=52560;eval=52561;seed=8364640122973076249;
[3]	3 days ago	00:00:00	> iter=52100;eval=52101;seed=10450884331206695397;
[4]	3 days ago	00:00:00	> iter=43510;eval=43511;seed=15322852344410324979;
[5]	3 days ago	00:00:00	> iter=69010;eval=69011;seed=7946725581494022999;
[6]	3 days ago	00:00:00	> iter=57640;eval=57641;seed=5376056338251316545;
[7]	3 days ago	00:00:00	> iter=56170;eval=56171;seed=14048048055183702911;

[0] Start a new run (ignore existing)

> Select checkpoint index:

Persistencia Resiliente

- **Identificación única:** Búsqueda automática basada en el hash del estado actual.
- **Bifurcación de control:** Carga automática si hay un checkpoint, o selección interactiva si hay múltiples.
- **Limpieza al terminar:** El archivo de estado en disco se elimina al completar con éxito.

Carga Extensible: Generación de Problemas desde Archivos

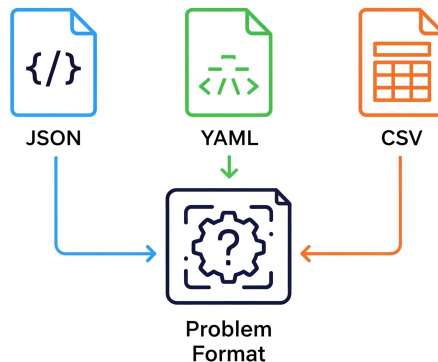
Integración y Flexibilidad

Importación Directa: Automatiza el pipeline de experimentación permitiendo la inyección dinámica de instancias complejas sin modificar código fuente.

JSON **YAML** **CSV**

Estructuras Admitidas:

- Matrices de distancia simétricas/asimétricas.
- Coordenadas de nodos para cálculo Euclidiano.
- Restricciones de capacidad y ventanas temporales.



COMPATIBILIDAD CON ENTORNOS CIENTÍFICOS: Serialización ultrarrápida. Permite exportar resultados intermedios y soluciones finales en formatos estándar para su posterior tratamiento analítico en herramientas de Data Science.

Una Solución Robusta, Flexible y Extensible

Rendimiento

Ventajas de Rust

El diseño de roma demuestra la viabilidad y enormes ventajas computacionales de emplear Rust para la optimización sin dependencias externas.

Un sistema robusto, seguro en memoria y con tiempos de ejecución altamente competitivos frente a alternativas en C++ o Python.

Ecosistema Nativo

El Rol de Cargo

Aprovechamiento integral de las herramientas del lenguaje, destacando el papel fundamental de **cargo** como gestor de proyectos.

Permite consolidar una documentación técnica completa y navegable directamente mediante **cargo doc**.



Distribución

Publicación y Uso

La biblioteca roma ha sido empaquetada formalmente como un **crate** de Rust, simplificando su importación en aplicaciones de terceros.

The screenshot shows the official crates.io page for the `roma_lib` crate, version 0.1.2. The page is titled "roma_lib v0.1.2" and describes it as "A Rust metaheuristics framework inspired by jMetal for optimization and experimentation." It includes navigation links for "Readme", "Code", "3 Versions", "Dependencies", "Dependents", and "Security". The "Readme" tab is active, showing the project's purpose: "It provides reusable building blocks to define optimization problems, configure algorithms, run reproducible executions, and observe progress through console or file-based reports." The "Features" section lists: "Single-objective algorithms: Hill Climbing, Genetic Algorithm, Simulated Annealing, PSO", "Multi-objective optimization with NSGA-II and Pareto rank/crowding quality metadata", "Generic Problem and Solution abstractions for custom domains", "Structured observers (console, chart, HTML report) and checkpoint utilities", and "Experiment runner for repeated case execution and comparative summaries". The "Metadata" section shows the crate was published "about 1 month ago", is version "v1.0.0", is "MIT licensed", has a size of "15K Bz2", and a download size of "12K KiB". The "Install" section provides the command: `cargo add roma_lib`. The "Documentation" section links to `docs.rs/roma_lib/0.1.2`.

Limitaciones y Líneas Futuras

Modelado y Arquitectura Rust

El Reto de la Ausencia de Herencia

La falta de herencia clásica ha introducido las siguientes limitaciones de diseño:

- **Código repetido** e implementaciones que resultan costosas.
- **Sistemas muy rígidos** para el modelado de jerarquías.

Complejidad paralela

No se ha conseguido explotar la paralelización correctamente por la rigidez estructural y por su complejidad.

Líneas de Mejora

Evolución y Optimización del Sistema

Propuestas para escalar la robustez y rendimiento de la biblioteca:

- **Macros procedurales:** Para evitar la duplicación de código, logrando implementaciones sencillas y menos verbosas.
- **Mejorar paralelismo:** Rediseñar flujos para explotar de forma óptima el procesamiento concurrente.
- **Filosofía Zero-Copy:** Minimizar costes de asignación y duplicado de datos científicos en memoria usando referencias.

Final de la Presentación

Grado en Ingeniería del Software

¡Muchas Gracias!

Por vuestra atención y tiempo

Realizado por: **David Muñoz del Valle**

Tutorizado por: **Gabriel Jesús Luque Polo**

Convocatoria Julio 2026



**E.T.S. INGENIERÍA
INFORMÁTICA**

UNIVERSIDAD DE MÁLAGA



Bienvenido a **roma**, una biblioteca robusta y extensible diseñada para la optimización de problemas complejos.